

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO1 – limitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	S.Sri Sai Prasanna Durga	Reg. No.:	192424241
Section / Batch:	2024	Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

$q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$

Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions**3. Problem Statement**

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Example Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

User Menu

1.Search Date

2.Search Phone Number

3.Search Hashtag

4.Search Mention

5.Search Prefix

6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Intelligent Email, Password & Mobile Validator

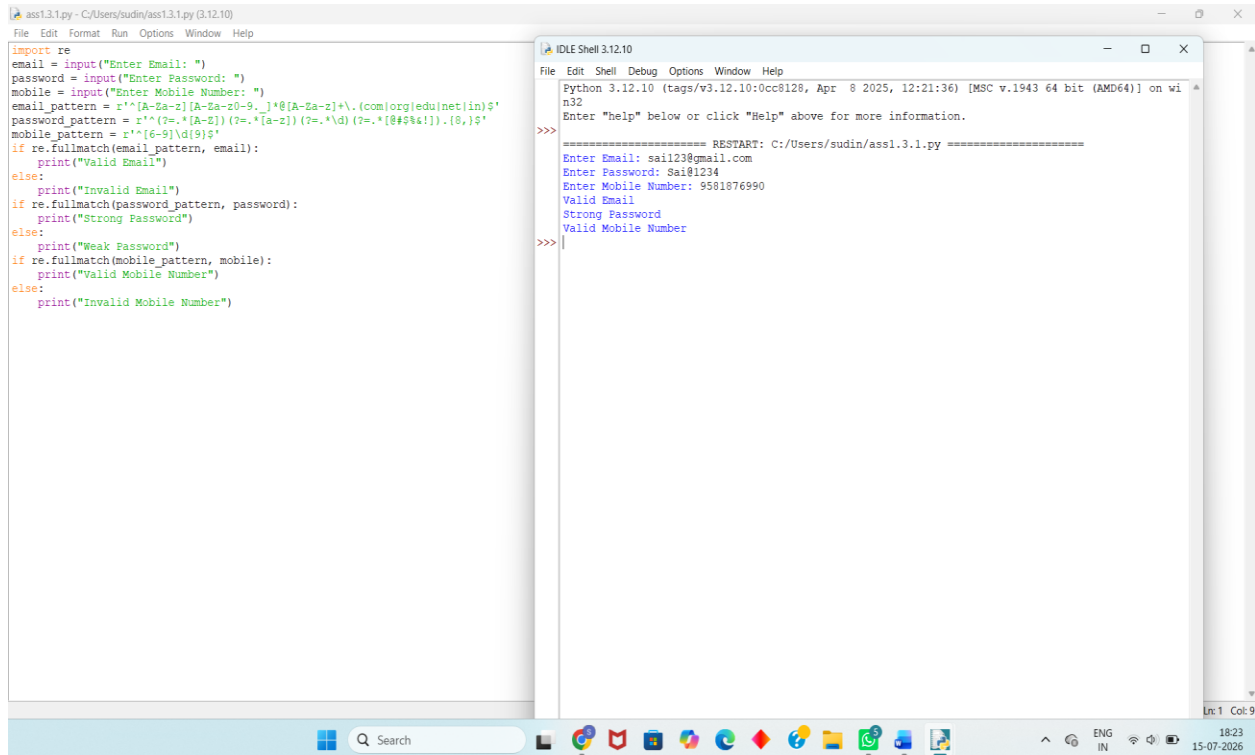
Algorithm:

1. Import the re module.
2. Read email, password, and mobile number from the user.
3. Validate each input using Regular Expressions.
4. Check whether each input matches the required pattern.
5. Display the validation result.

Code:

```
import re
email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!]).{8,}$'
mobile_pattern = r'^[6-9]\d{9}$'
if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")
if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Output:



The image shows a screenshot of a Python script and its execution output. The script, located at C:/Users/sudin/ass1.3.1.py, uses regular expressions to validate email, password, and mobile number inputs. The output window shows the program's execution, including prompts for email, password, and mobile number, and the resulting validation status for each.

```
ass1.3.1.py - C:/Users/sudin/ass1.3.1.py (3.12.10)
File Edit Format Run Options Window Help

import re
email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")
email_pattern = r'@[A-Za-z]{1,3}\.[a-zA-Z]{2,4}\.[a-zA-Z]{2,4}$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$%&!]).{8,}$'
mobile_pattern = r'^[6-9]\d{9}$'
if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")
if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")

IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sudin/ass1.3.1.py =====
Enter Email: sai123@gmail.com
Enter Password: Sai1234
Enter Mobile Number: 9581876990
Valid Email
Strong Password
Valid Mobile Number
>>>
```

Question 2: Finite State Automata (DFA) Simulator

Algorithm:

1. Define DFA states and transitions.
2. Read the input string.
3. Start from the initial state.
4. Move to the next state for each character.
5. Display the transition path.
6. If the final state is reached, print Accepted; otherwise Rejected.

Code:

```
transitions = {
    'q0': {'a': 'q1', 'b': 'q0'},
    'q1': {'a': 'q1', 'b': 'q2'},
    'q2': {'a': 'q1', 'b': 'q0'}
}

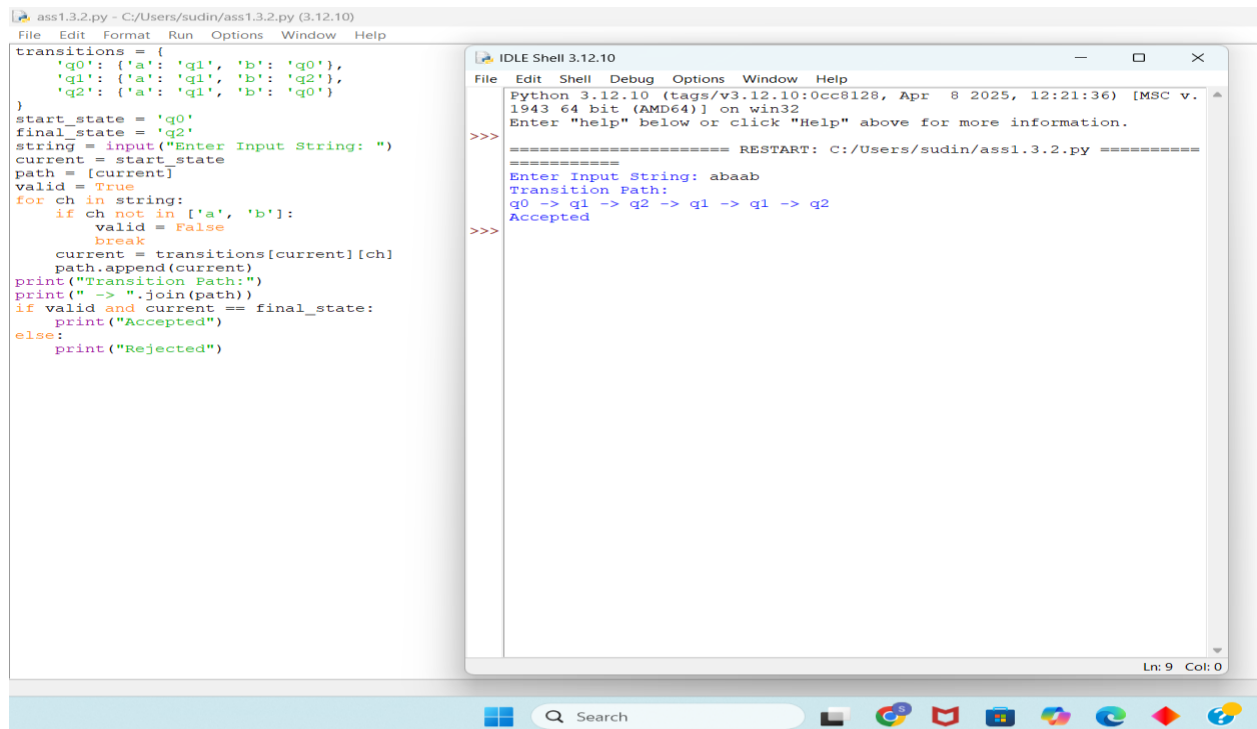
start_state = 'q0'
final_state = 'q2'
```

```

string = input("Enter Input String: ")
current = start_state
path = [current]
valid = True
for ch in string:
    if ch not in ['a', 'b']:
        valid = False
        break
    current = transitions[current][ch]
    path.append(current)
print("Transition Path:")
print(" -> ".join(path))
if valid and current == final_state:
    print("Accepted")
else:
    print("Rejected")

```

Output:



The screenshot shows a Python IDE with two windows. The left window displays the source code for a finite state automaton (FSA) simulation. The right window shows the execution output in the IDLE Shell.

Source Code (Left Window):

```

transitions = {
    'q0': {'a': 'q1', 'b': 'q0'},
    'q1': {'a': 'q1', 'b': 'q2'},
    'q2': {'a': 'q1', 'b': 'q0'}
}
start_state = 'q0'
final_state = 'q2'
string = input("Enter Input String: ")
current = start_state
path = [current]
valid = True
for ch in string:
    if ch not in ['a', 'b']:
        valid = False
        break
    current = transitions[current][ch]
    path.append(current)
print("Transition Path:")
print(" -> ".join(path))
if valid and current == final_state:
    print("Accepted")
else:
    print("Rejected")

```

Execution Output (Right Window):

```

Python 3.12.10 (tags/v3.12.10:0cc8128, Apr  8 2025, 12:21:36) [MSC v.
1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/sudin/ass1.3.2.py =====
Enter Input String: abaab
Transition Path:
q0 -> q1 -> q2 -> q1 -> q1 -> q2
>>> Accepted

```

Question 3: Smart Pattern Matching Engine

Algorithm:

1. Import the re module.
2. Read the text from the user.
3. Display the search menu.
4. Apply the selected Regular Expression.
5. Find all matching patterns.
6. Display the result.

Code:

```
import re
text = input("Enter Text: ")
while True:
    print("\n----- MENU -----")
    print("1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Exit")
    choice = input("Enter Choice (1-7): ")
    if choice == "1":
        result = re.findall(r"\b\d{2}^\d{2}^\d{4}\b", text)
        print("Dates Found:", result)
    elif choice == "2":
        result = re.findall(r"\b[6-9]\d{9}\b", text)
        print("Phone Numbers Found:", result)
    elif choice == "3":
        result = re.findall(r"#\w+", text)
        print("Hashtags Found:", result)
```

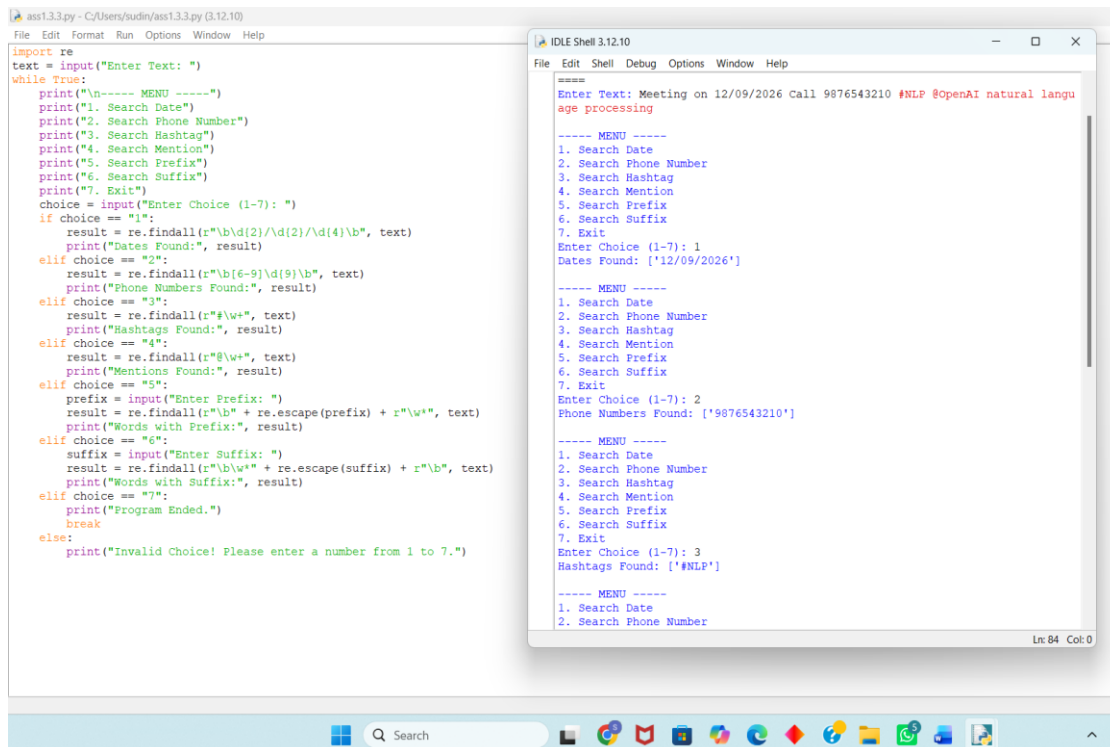


```

elif choice == "4":
    result = re.findall(r"@w+", text)
    print("Mentions Found:", result)
elif choice == "5":
    prefix = input("Enter Prefix: ")
    result = re.findall(r"\b" + re.escape(prefix) + r"w*", text)
    print("Words with Prefix:", result)
elif choice == "6":
    suffix = input("Enter Suffix: ")
    result = re.findall(r"\b" + re.escape(suffix) + r"\b", text)
    print("Words with Suffix:", result)
elif choice == "7":
    print("Program Ended.")
    break
else:
    print("Invalid Choice! Please enter a number from 1 to 7.")

```

Output:



The screenshot shows a Python IDE with two windows. The left window displays the source code for a program that searches for various patterns in a text string. The right window shows the output of the program, which includes a menu, user input, and the results of the searches.

```

ass1.3.3.py - C:/Users/sudin/ass1.3.3.py (3.12.10)
File Edit Shell Debug Options Window Help
import re
text = input("Enter Text: ")
while True:
    print("\n----- MENU -----")
    print("1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Exit")
    choice = input("Enter Choice (1-7): ")
    if choice == "1":
        result = re.findall(r"\b\d{2}/\d{2}/\d{4}\b", text)
        print("Dates Found:", result)
    elif choice == "2":
        result = re.findall(r"\b[6-9]\d{9}\b", text)
        print("Phone Numbers Found:", result)
    elif choice == "3":
        result = re.findall(r"#\w+", text)
        print("Hashtags Found:", result)
    elif choice == "4":
        result = re.findall(r"@w+", text)
        print("Mentions Found:", result)
    elif choice == "5":
        prefix = input("Enter Prefix: ")
        result = re.findall(r"\b" + re.escape(prefix) + r"w*", text)
        print("Words with Prefix:", result)
    elif choice == "6":
        suffix = input("Enter Suffix: ")
        result = re.findall(r"\b" + re.escape(suffix) + r"\b", text)
        print("Words with Suffix:", result)
    elif choice == "7":
        print("Program Ended.")
        break
    else:
        print("Invalid Choice! Please enter a number from 1 to 7.")

```

```

=====
Enter Text: Meeting on 12/09/2026 Call 9876543210 #NLP @OpenAI natural langu
age processing
----- MENU -----
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter Choice (1-7): 1
Dates Found: ['12/09/2026']

----- MENU -----
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter Choice (1-7): 2
Phone Numbers Found: ['9876543210']

----- MENU -----
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Exit
Enter Choice (1-7): 3
Hashtags Found: ['#NLP']

----- MENU -----
1. Search Date
2. Search Phone Number

```

